



**Maynooth
University**

National University
of Ireland Maynooth

**SEMESTER 1
2023-2024**

**CS264FZ
Software Design**

Dr. Guo Kun, Dr. Joe Timoney, Dr. Michael English

Time allowed: 2 hours

Answer all *four* questions

All questions carry equal marks

Instructions

	Yes	No
Log Books Allowed	☐	☒
Formula Tables Allowed	☐	☒
Other Allowed (<i>enter details</i>)	☐	☒

General (*enter details*)

QUESTION 1

- (a) Please give an example to explain how the state pattern is used in the real world. (5 marks)
- (b) Can extrinsic data of flyweights be shareable? How do clients handle the data? (5 marks)
- (c) When should we use the interpreter pattern? Please give an example. (5 marks)
- (d) Can we use different types of observers in the same application? Please give your reason. (5 marks)
- (e) How are the abstraction and implementation implemented in the bridge pattern? You can use classes ElectronicGoods and Sate in the textbook as an example. (5 marks)

QUESTION 2

- (a) Suppose there is an interface Employee and its two concrete subclasses, Worker and Manager. Which pattern can be used to add a new operation, such as getSalary(), to classes Worker and Manager without modifying them? Please give your solution. (5 marks)
- (b) Please describe the differences and relationships between the mediator and observer patterns. (5 marks)
- (c) Please explain the differences between the eager and lazy initializations in the singleton pattern. (5 marks)
- (d) Can the iterator pattern be used with the composite pattern? Please give your reason. (5 marks)
- (e) Please describe a solution to handle the situation in the chain-of-responsibility pattern where the request cannot be handled by any of the handlers in the chain. (5 marks)

QUESTION 3

- (a) It is necessary to create character classes and strategy interfaces when the strategy pattern is used to simulate attacks of different characters in a game. The Test class and the program's output are as follows :

```
// this is some code
public class StrategyPatternExample {
    public static void main(String[] args) {
        Character player = new Character();
        Character enemy = new Character();
        AttackStrategy magicalAttack = new
MagicalAttackStrategy ();
        AttackStrategy physicalAttack = new
PhysicalAttackStrategy ();
        player.setAttackStrategy(magicalAttack);
        enemy.setAttackStrategy(physicalAttack);
        player.attack(enemy);
        enemy.attack(player);
    }
}
```

Outputs:

magical attack

physical attack

Please finish the following tasks.

- (i) Create an attack strategy interface `AttackStrategy` with an `attack()` method. (5 marks)
- (ii) Create two attack strategy implementation classes, namely `MagicalAttackStrategy` and `PhysicalAttackStrategy`. (10 marks)
- (iii) Create a character class `Character` to implement different attack strategies. (10 marks)

QUESTION 4

- (a) Suppose you are designing an e-commerce platform that needs to integrate multiple third-party payment gateways, including Alipay, WeChatPay, and UnionPay. Please use the facade pattern to design a unified payment interface, allowing customers to choose and complete payment operations through a simple interface. The detailed requirements are as follows:
 - (i) Create three classes, namely `Alipay`, `WechatPay`, and `UnionPay`, to implement the three payment gateways, respectively. Each class contains a `payment()` method. (8 marks)
 - (ii) Write a facade class `PaymentFacade` to encapsulate the payment operations of the three payment classes. (12 marks)
 - (iii) Write a `Test` class to test the functions of the above classes. (5 marks)